



Quick Sort

QuickSort is a widely used divide-and-conquer sorting algorithm. It works by selecting a pivot element from the list and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then recursively sorted, and the process continues until the entire list is sorted.

Here's a step-by-step explanation of how the QuickSort algorithm works:

1. Choose a pivot element from the list. The pivot can be selected in various ways, such as taking the first element, the last element, or a random element. For simplicity, let's assume we select the last element of the list as the pivot.
2. Partition the list by rearranging the elements such that all elements smaller than the pivot are placed before it, and all elements greater than the pivot are placed after it. After this partitioning step, the pivot is in its final sorted position.
3. Recursively apply the above two steps to the sub-array of elements smaller than the pivot and the sub-array of elements greater than the pivot. This means performing the partitioning step on each sub-array and continuing the process until each sub-array contains only one element or is empty.
4. Once the recursion ends and all sub-arrays are sorted, the entire list is sorted.

Here's an example to illustrate the steps:

Consider the list: [8, 3, 1, 7, 0, 10, 2]

Step 1: Choose the pivot. Let's choose the last element, 2.

Step 2: Partition the list. Rearrange the elements such that all elements smaller than 2 come before it, and all elements greater than 2 come after it.

Partitioned list: [1, 0, 2, 7, 8, 10, 3]

Step 3: Recursively apply the above steps to the sub-arrays. In this case, we have two sub-arrays: [1, 0] and [7, 8, 10, 3].

For the sub-array [1, 0]:

- Choose the last element, 0, as the pivot.
- Partition the list: [0, 1]

For the sub-array [7, 8, 10, 3]:

- Choose the last element, 3, as the pivot.
- Partition the list: [7, 8, 3, 10]

Continue applying the steps recursively to the sub-arrays until each sub-array contains only one element or is empty.

Step 4: The recursion ends when each sub-array is sorted (contains only one element or is empty). At this point, the entire list is sorted.

Final sorted list: [0, 1, 2, 3, 7, 8, 10]

QuickSort has an average time complexity of $O(n \log n)$, making it efficient for large lists. However, its worst-case time complexity can be $O(n^2)$ if the pivot selection is unbalanced, which can be mitigated by using randomized pivot selection or other techniques. Overall, QuickSort is a widely used and efficient sorting algorithm.

```
1. public class QuickSort {
2.     public static void main(String[] args) {
3.         int[] arr = {8, 3, 1, 7, 0, 10, 2};
4.         quickSort(arr, 0, arr.length - 1);
5.         System.out.println("Sorted array:");
6.         for (int num : arr) {
7.             System.out.print(num + " ");
8.         }
9.     }
10.    public static void quickSort(int[] arr, int low, int
        high) {
11.        if (low < high) {
12.            // Partition the array
13.            int partitionIndex = partition(arr, low,
                high);
14.            // Recursively sort the sub-arrays
15.            quickSort(arr, low, partitionIndex - 1);
16.            quickSort(arr, partitionIndex + 1, high);
17.        }
18.    }
```

```
1.    public static int partition(int[] arr, int low, int high) {
2.        int pivot = arr[high];
3.        int i = low - 1;
4.        for (int j = low; j < high; j++) {
5.            if (arr[j] <= pivot) {
6.                i++;
7.                // Swap arr[i] and arr[j]
8.                int temp = arr[i];
9.                arr[i] = arr[j];
10.               arr[j] = temp;
11.            }
12.        }
13.        int temp = arr[i + 1];
14.        arr[i + 1] = arr[high];
15.        arr[high] = temp;
16.        return i + 1;
17.    }
18. }
```

Selection Sort

Certainly! Here's a step-by-step explanation of how the selection sort algorithm works:

1. Start with an unsorted list of elements.

Example: [64, 25, 12, 22, 11]

2. The algorithm divides the list into two parts: the sorted part and the unsorted part. Initially, the sorted part is empty, and the unsorted part contains all the elements.

3. In each iteration, the algorithm finds the minimum element from the unsorted part of the list.

4. To find the minimum element, the algorithm compares each element in the unsorted part with the current minimum element. If it finds a smaller element, it updates the minimum element.

5. Once the minimum element is found, it is swapped with the first element of the unsorted part. This places the minimum element at the end of the sorted part and expands the sorted part by one element.

6. The algorithm repeats steps 3-5 until the entire list is sorted. The sorted part gradually grows from left to right until it encompasses the entire list.

7. Finally, when the algorithm completes all iterations, the list is fully sorted.

Here's an example to illustrate the steps:

Iteration 1:

- Find the minimum element from the unsorted part [64, 25, 12, 22, 11]. The minimum is 11.
- Swap the minimum element (11) with the first element (64) of the unsorted part.
- The sorted part becomes [11], and the unsorted part becomes [64, 25, 12, 22].

Iteration 2:

- Find the minimum element from the unsorted part [64, 25, 12, 22]. The minimum is 12.
- Swap the minimum element (12) with the first element (64) of the unsorted part.
- The sorted part becomes [11, 12], and the unsorted part becomes [64, 25, 22].

Iteration 3:

- Find the minimum element from the unsorted part [64, 25, 22]. The minimum is 22.
- Swap the minimum element (22) with the first element (64) of the unsorted part.
- The sorted part becomes [11, 12, 22], and the unsorted part becomes [64, 25].

Iteration 4:

- Find the minimum element from the unsorted part [64, 25]. The minimum is 25.
- Swap the minimum element (25) with the first element (64) of the unsorted part.
- The sorted part becomes [11, 12, 22, 25], and the unsorted part becomes [64].

Iteration 5:

- Find the minimum element from the unsorted part [64]. The minimum is 64.
- Swap the minimum element (64) with the first element (64) of the unsorted part. (No change in this step)
- The sorted part becomes [11, 12, 22, 25, 64], and the unsorted part becomes [] (empty).

The iterations are now complete, and the list [64, 25, 12, 22, 11] is sorted in ascending order to [11, 12, 22, 25, 64].

Selection sort has a time complexity of $O(n^2)$, where n is the number of elements in the list. It is not very efficient for large lists, but it is simple to implement and performs well for small lists or partially sorted lists.

```

public class SelectionSort {
    public static void main(String[] args) {
        int[] arr = {64, 25, 12, 22, 11};
        selectionSort(arr);
        System.out.println("Sorted array:");
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    public static void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }
int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
    }
}

```



THANK YOU